CaptionAI: AI-Powered Social Media Caption Generator

Project Description:

Rising Waters is an AI-powered Flood Prediction System that predicts the likelihood of floods using historical rainfall and weather parameters. The application uses a Random Forest Machine Learning model trained on rainfall datasets to provide accurate flood risk predictions.

The system is built using Python, Flask, HTML, CSS, and Scikit-learn. Users enter weather parameters such as temperature, humidity, cloud cover, rainfall, seasonal rainfall values, and subdivision information. The trained model analyzes these inputs and predicts whether flooding is likely. The application provides a simple web interface and delivers real-time predictions for disaster preparedness and early warning.

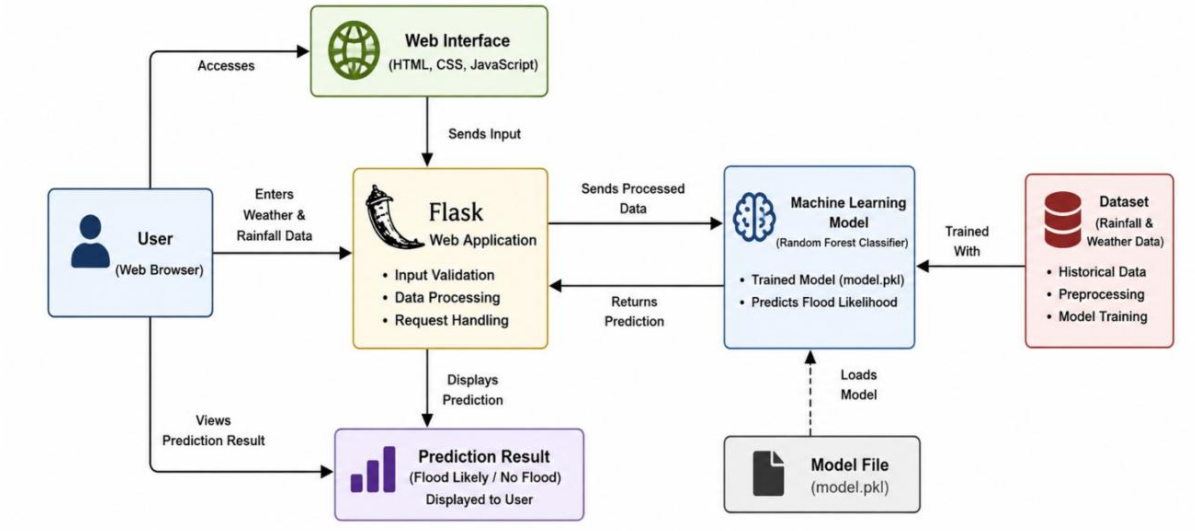
Scenarios

Scenario 1:A disaster management officer enters rainfall and weather conditions for a district. The system predicts whether flooding is likely so emergency response teams can prepare in advance.

Scenario 2:A weather analyst enters seasonal rainfall statistics to assess flood risk before the monsoon season and supports planning activities.

Scenario 3: A local authority uses current environmental data to estimate flood probability and issue alerts to nearby communities.

Technical Architecture:



Description: The system follows a three-layer architecture consisting of a web interface, Flask backend, and machine learning prediction engine. Users submit rainfall and weather information through the web application. Flask validates the inputs and forwards them to the trained Random Forest model. The model predicts whether flooding is likely, and the result is displayed instantly through the web interface.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- Python 3.10 or above
- Flask
- Scikit-learn
- Pandas
- NumPy
- Joblib
- HTML
- CSS
- VS Code
- Google Chrome

Project Workflow:

Milestone 1: Data Collection

- Collect rainfall dataset.
- Analyze dataset.
- Prepare training data.

Milestone 2: Model Development

- Train Random Forest model.
- Evaluate prediction accuracy.
- Save trained model (.pkl).

Milestone 3: Backend Development

- Develop Flask application.
- Load trained model.
- Implement prediction API

Milestone 4: Frontend Development

- Design prediction webpage.
- Connect frontend with Flask backend.
- Display prediction results.

Milestone 5: Testing & Deployment

- Test prediction functionality.
- Fix bugs.
- Deploy application

Milestone 6: Project Completion

- Final testing.
- Documentation.
- Project submission.

Milestone 1: Model Selection and Architecture

In this milestone, we focused on collecting historical rainfall and weather datasets, selecting an appropriate Machine Learning algorithm, and designing the overall system architecture for flood prediction. This phase established the foundation for building an accurate and reliable prediction system.

Activity 1.1: Collect Rainfall Dataset

Collect a historical rainfall dataset containing weather and rainfall parameters required for flood prediction.

Step 1 — Obtain Dataset

Download a historical rainfall dataset from a trusted source such as Kaggle or the India Meteorological Department (IMD).

Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.8667	649.9	0
28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3	256.4	1
28	75	42	3271.2	21.7	336.3	2343	570.1	186.2	308.9	0
29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.0667	862.5	0
31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4	586.9	0
30	70	38	2708	34.1	230	1943.1	500.8	138.3	254.1	0
29	74	40	3671.1	23.7	328	2737.8	581.7	256.9667	669.5	1
30	78	36	2648.3	28.8	283.7	2023.6	312.2	197.5333	450	0
30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.9	231.5	0
30	70	34	2848.6	28.4	296.7	1886.5	637	226.6667	531.2	0
28	73	30	2726.7	7.3	249.7	1934	535.7	330	809.4	0
28	77	40	3451.3	16.9	351.1	2453.1	630.2	316.0667	730.9	1
28	79	32	2610.8	8.3	295.2	1729	578.3	180.5667	342.9	0
31	72	35	2899.1	7.6	215	2066.1	610.5	188.4333	401.1	0
30	73	33	3024.5	40.4	303.1	2167	514	232.0333	541.6	0
28	76	31	2945.3	7.8	303.4	2170.2	463.9	306.7333	721.2	0
28	75	31	2704.8	50.5	240.4	1851.7	562.1	234.5667	580.8	0
31	77	33	2501.9	47.9	767	1104.3	582.6	154.7667	218.7	0
28	71	43	3003.3	49.2	346.8	2025	582.3	212.2667	389.8	0
28	77	31	3303.1	40.6	283.7	2318.2	660.6	321.4333	876.6	0
31	79	43	2719.9	47.8	290.3	1927.5	454.3	163.0333	385	0
28	74	36	3267.6	51.9	399.4	2231.2	585.1	221.0333	369.5	0
28	70	30	3484.7	25.3	202.3	2928.4	328.6	240.8333	642.5	1
30	71	41	4226.4	22.2	363	3451.3	389.9	337.2333	826.3	1
31	72	36	3062.1	20.5	428.5	1995.2	617.9	229.6	430.6	0
30	70	38	2965.4	34.4	301.5	2307.8	321.7	187.9667	341.3	0
28	72	40	2994.7	54.1	401.4	2258.9	280.2	240.0667	454.8	0
31	79	38	2502.8	78.6	254.3	1640.4	529.4	196.9	508.8	0
31	77	36	3361.6	42.6	417.6	2353.5	548	315.5333	798.6	0
30	70	38	3018	21.6	546.5	1719.7	730.2	211.0333	228.2	0
30	73	30	3259.6	3.6	277.8	2558	420.1	180.5667	410	1
29	72	37	3403	19.4	788.1	1797.8	797.7	113.6667	305.5	0
31	75	42	4072.9	10.3	915.2	2581.9	565.5	286.4333	120.5	1
31	76	42	2410.7	76.2	246.8	1653.5	434.2	284.3	746.2	0
31	72	41	2498.2	32.3	195.4	1682.9	587.6	143.7667	374.7	0
30	73	35	3043.3	17.7	616.6	1947.5	461.5	206.9333	154.3	0
29	72	33	2818.2	27.7	371.3	1877.1	542	161.8667	348.5	0
31	70	36	2634.1	79.4	397.3	1841.3	316.1	227.2	502	0
29	74	39	2937.5	17.2	302.8	1969.4	648.1	208.6	520.7	0

Step 2 — Analyze Dataset

Open the dataset in Excel or Python and verify all required features.

Example attributes:

- Temperature
- Humidity
- Cloud Cover
- Annual Rainfall
- Jan–Feb Rainfall
- Mar–May Rainfall
- Jun–Sep Rainfall
- Oct–Dec Rainfall
- Average June Rainfall
- Subdivision
- Flood (Target)

A	B	C	D	E	F	G	H	I	J	K
29	74	39	2937.5	17.2	302.8	1969.4	648.1	208.6	520.7	0
29	74	35	3117.8	2.1	347.6	2162.7	605.3	202.1333	389	0
30	78	34	3111.1	20.5	532	2031.8	526.7	265.8667	380.1	0
31	79	31	3050.9	7	395.4	2071.4	577	271.2	621.7	0
28	70	32	3464.2	98.1	624.3	2067	674.9	264.8333	316.1	0
28	79	42	2490	33	336	1498.7	622.3	166.3	286.2	0
28	74	30	2432.4	14.3	170.2	1716.5	531.4	183.2667	496.4	0
28	72	42	3565.5	7.3	331.2	2485.7	741.3	306.3333	836	1
30	74	41	2998.1	49.5	325.9	2359.5	263.2	185.3667	470.4	0
30	73	34	3039.2	51.3	385.5	2183.7	418.7	303.4	697.9	0
30	75	32	2942.6	2.4	542.6	2094.7	302.8	178.7667	96.3	0
31	77	39	3146.6	53.8	341.5	2342.9	408.4	212.7667	396.3	0
30	71	43	2705.5	13.1	366	1823	503.5	258.0333	625.6	0
31	70	41	2334.8	53.4	347.6	1477.7	456.1	192.2333	362.1	0
28	74	33	2544.9	35.7	206.3	1825.1	477.8	113.5	285.1	0
30	72	30	2938	26.3	407	2107.4	397.2	266.1	618.8	0
30	76	36	3134.7	10.5	698.3	1849.7	576.1	260.8	238.2	0
29	76	32	2798.4	19.6	518	1720.1	540.6	251.8	404.1	0
31	71	34	3103.3	17.2	477.2	2107.4	501.5	290.6667	490.8	0
29	77	31	2923.1	23.7	545.9	1948.7	404.8	237.7667	359.8	0
29	74	35	3746	24.3	504.2	2831.2	386.3	290.9333	525.6	1
30	71	39	3385.5	17.8	791	1939.2	637.5	160.1	59.7	0
30	72	30	4257.8	45	606	3229.3	377.6	335.0667	504.7	1
29	70	44	3375.8	83.1	607.1	2101.6	584	81.63333	227.5	0
29	71	35	2651.1	55	323.2	1848.5	424.4	131.1	236.2	0
29	79	36	2869.1	8.8	245.2	2079.8	535.2	126.4667	284.6	0
31	71	36	2342.4	10	352.6	1508.9	470.9	199.2333	383.2	0
31	77	41	2621.7	9.9	330.3	1593.3	688.2	165.4	401	0
28	71	44	2569.1	14.4	339.6	1937.3	277.8	180.5667	296.8	0
29	76	33	3392.7	37.8	312.5	2711.4	331	232.1333	606.4	1
28	78	40	2665	9.1	364	1870.6	421.3	183.5	323.1	0
31	78	41	2703.5	30.5	447.7	1860.7	364.6	178.4333	246.2	0
31	73	31	3076.8	50.1	450.6	2254.6	321.5	296.5333	572.1	0
29	77	40	2739.4	10.2	526	1596.8	606.4	133.9333	34.2	0
29	75	44	2412.5	0.3	263.8	1749.2	399.2	205.6667	497.1	0
30	78	40	2767.4	7	365.6	2188.2	206.6	88.96667	45.4	0
30	72	35	3498.4	26.6	349.3	2529.3	593.1	288.1333	702.2	1
30	73	34	2068.8	1.6	231.3	1297.1	538.8	65.6	121	0
30	72	37	3047.6	16.8	436.9	1788.5	805.4	199.8667	293.2	0
30	70	38	3176.7	18	502.1	2081.1	575.6	252.7	361.3	0

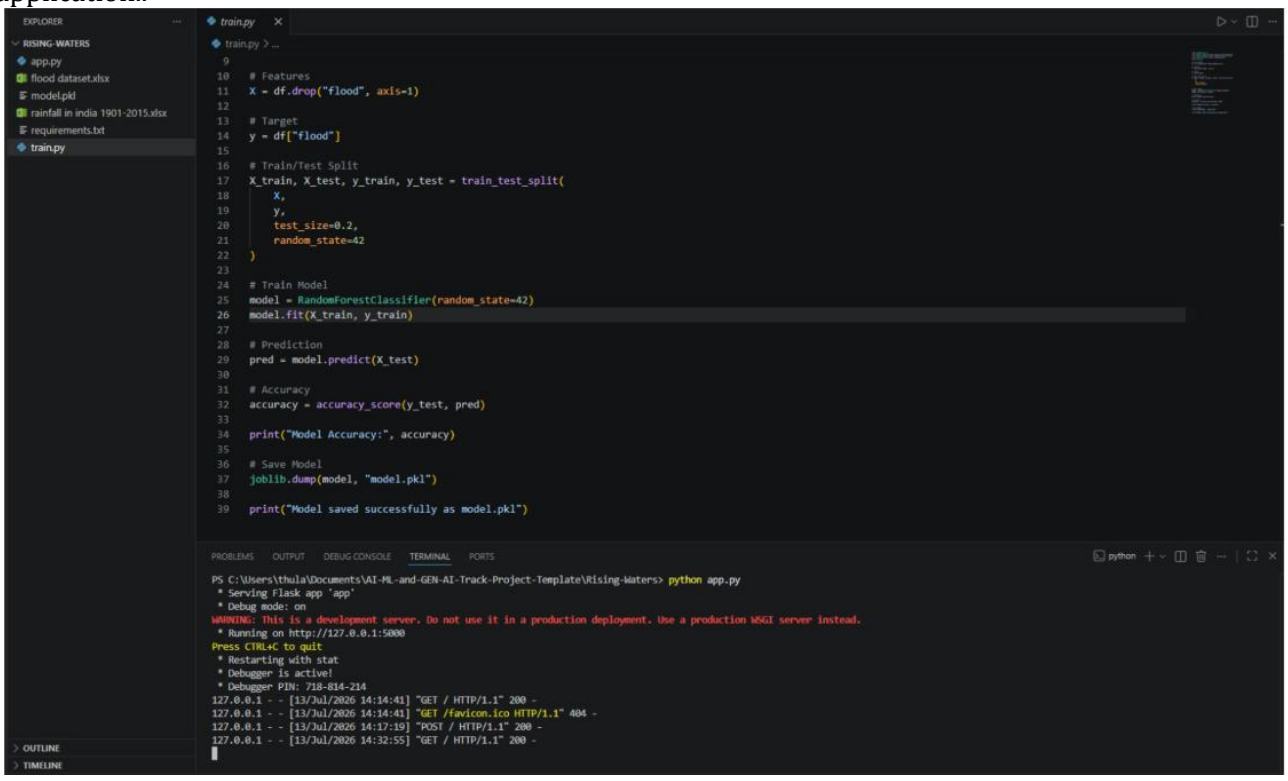
.

Step 3 — Load the Dataset: Open the flood dataset.xlsx file and verify that it contains all required features such as Temperature, Humidity, Cloud Cover, Annual Rainfall, seasonal rainfall values, Average June rainfall, Subdivision, and the target Flood column.

A1 | Temp

	A	B	C	D	E	F	G	H	I	J	K
1	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
2	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.8667	649.9	0
3	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.3	256.4	1
4	28	75	42	3271.2	21.7	336.3	2343	570.1	186.2	308.9	0
5	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.0667	862.5	0
6	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.4	586.9	0
7	30	70	38	2708	34.1	230	1943.1	500.8	138.3	254.1	0
8	29	74	40	3671.1	23.7	328	2737.8	581.7	256.9667	669.5	1
9	30	78	36	2648.3	28.8	283.7	2023.6	312.2	197.5333	450	0
10	30	71	40	3050.2	65.9	628.3	1940.4	415.5	234.9	231.5	0
11	30	70	34	2848.6	28.4	296.7	1886.5	637	226.6667	531.2	0
12	28	73	30	2726.7	7.3	249.7	1934	535.7	330	809.4	0
13	28	77	40	3451.3	16.9	351.1	2453.1	630.2	316.0667	730.9	1
14	28	79	32	2610.8	8.3	295.2	1729	578.3	180.5667	342.9	0
15	31	72	35	2899.1	7.6	215	2066.1	610.5	188.4333	401.1	0
16	30	73	33	3024.5	40.4	303.1	2167	514	232.0333	541.6	0
17	28	76	31	2945.3	7.8	303.4	2170.2	463.9	306.7333	721.2	0
18	28	75	31	2704.8	50.5	240.4	1851.7	562.1	234.5667	580.8	0
19	31	77	33	2501.9	47.9	767	1104.3	582.6	154.7667	218.7	0
20	28	71	43	3003.3	49.2	346.8	2025	582.3	212.2667	389.8	0
21	28	77	31	3303.1	40.6	283.7	2318.2	660.6	321.4333	876.6	0
22	31	79	43	2719.9	47.8	290.3	1927.5	454.3	163.0333	385	0
23	28	74	36	3267.6	51.9	399.4	2231.2	585.1	221.0333	369.5	0
24	28	70	30	3484.7	25.3	202.3	2928.4	328.6	240.8333	642.5	1
25	30	71	41	4226.4	22.2	363	3451.3	389.9	337.2333	826.3	1
26	31	72	36	3062.1	20.5	428.5	1995.2	617.9	229.6	430.6	0
27	30	70	38	2965.4	34.4	301.5	2307.8	321.7	187.9667	341.3	0
28	28	72	40	2994.7	54.1	401.4	2258.9	280.2	240.0667	454.8	0
29	31	79	38	2502.8	78.6	254.3	1640.4	529.4	196.9	508.8	0
30	31	77	36	3361.6	42.6	417.6	2353.5	548	315.5333	798.6	0
31	30	70	38	3018	21.6	546.5	1719.7	730.2	211.0333	228.2	0
32	30	73	30	3259.6	3.6	277.8	2558	420.1	180.5667	410	1
33	29	72	37	3403	19.4	788.1	1797.8	797.7	113.6667	305.5	0
34	31	75	42	4072.9	10.3	915.2	2581.9	565.5	286.4333	120.5	1
35	31	76	42	2410.7	76.2	246.8	1653.5	434.2	284.3	746.2	0
36	31	72	41	2498.2	32.3	195.4	1682.9	587.6	143.7667	374.7	0
37	30	73	35	3043.3	17.7	616.6	1947.5	461.5	206.9333	154.3	0
38	29	72	33	2818.2	27.7	371.3	1877.1	542	161.8667	348.5	0
39	31	70	36	2634.1	79.4	397.3	1841.3	316.1	227.2	502	0
40	29	74	39	2937.5	17.2	302.8	1969.4	648.1	208.6	520.7	0
41	29	74	35	3117.8	2.1	347.6	2162.7	605.3	202.1333	389	0

• **Step 4** — Prepare the Project: Verify that the project contains all required source files, trained model (model.pkl), dataset files, and dependency file (requirements.txt) before running the application..



Step 5 — Train the Machine Learning Model:
Run the training script using:
python train.py
This trains the Random Forest model and saves it as model.pkl.

Step 6 — Run the Web Application:

python app.py

Open the browser and visit:

http://127.0.0.1:5000

Enter the flood parameters and click Predict to view the prediction result.

Activity 1.2: Research and Select the Appropriate Machine Learning Model

Here are the activities performed to select the best machine learning model for the Flood Prediction project:

Understand Project Requirements: Analyze the objective of predicting flood occurrence using historical weather and rainfall data.
Explore Machine Learning Algorithms: Compare different classification algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine.
Evaluate Model Performance: Measure each model using Accuracy, Precision, Recall, and F1-Score on the testing dataset.
Select the Optimal Model: Choose the Random Forest Classifier because it provides higher prediction accuracy, better handling of multiple input features, and improved robustness against overfitting.

Activity 1.3: Define the Architecture of the Application

Design System Architecture: Create the application architecture consisting of the User Interface, Flask Backend, Machine Learning Model, and Prediction Output.

Frontend Functionality: Design an input form where users enter Temperature, Humidity, Cloud Cover, Annual Rainfall, seasonal rainfall values, Average June rainfall, and Subdivision.

Backend Responsibilities: The Flask backend validates user inputs, preprocesses the data, loads the trained Random Forest model (model.pkl), and performs flood prediction.

Model Integration: Integrate the trained machine learning model with the Flask application using the Joblib library. The prediction result ("Flood Likely" or "No Flood") is returned and displayed on the web page.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv

D:\projects>"d:\projects\venv\Scrpts\activate"

(myenv) D:\projects> pip install -r requiremen
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

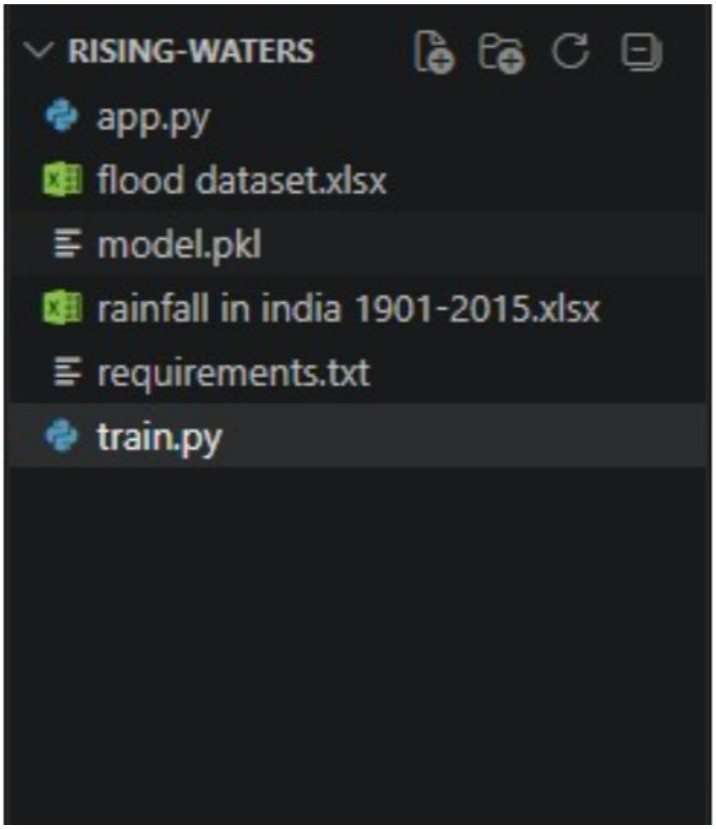
- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
  GROQ_API_KEY=your_groq_api_key_here
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Flood Prediction Features

Implement the main functionalities required for flood prediction using machine learning.

User Input Module: Accepts Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, Average June Rainfall, and Subdivision as input from the user.

Data Validation: Validates all input values to ensure they are numeric, complete, and within acceptable ranges before prediction.

Flood Prediction Module: Uses the trained Random Forest Classifier (model.pkl) to analyze the input features and predict whether a flood is likely to occur.

Prediction Result Display: Displays the prediction result as "Flood Likely" or "No Flood" on the web interface.

Activity 2.2: Implement the Flask Backend

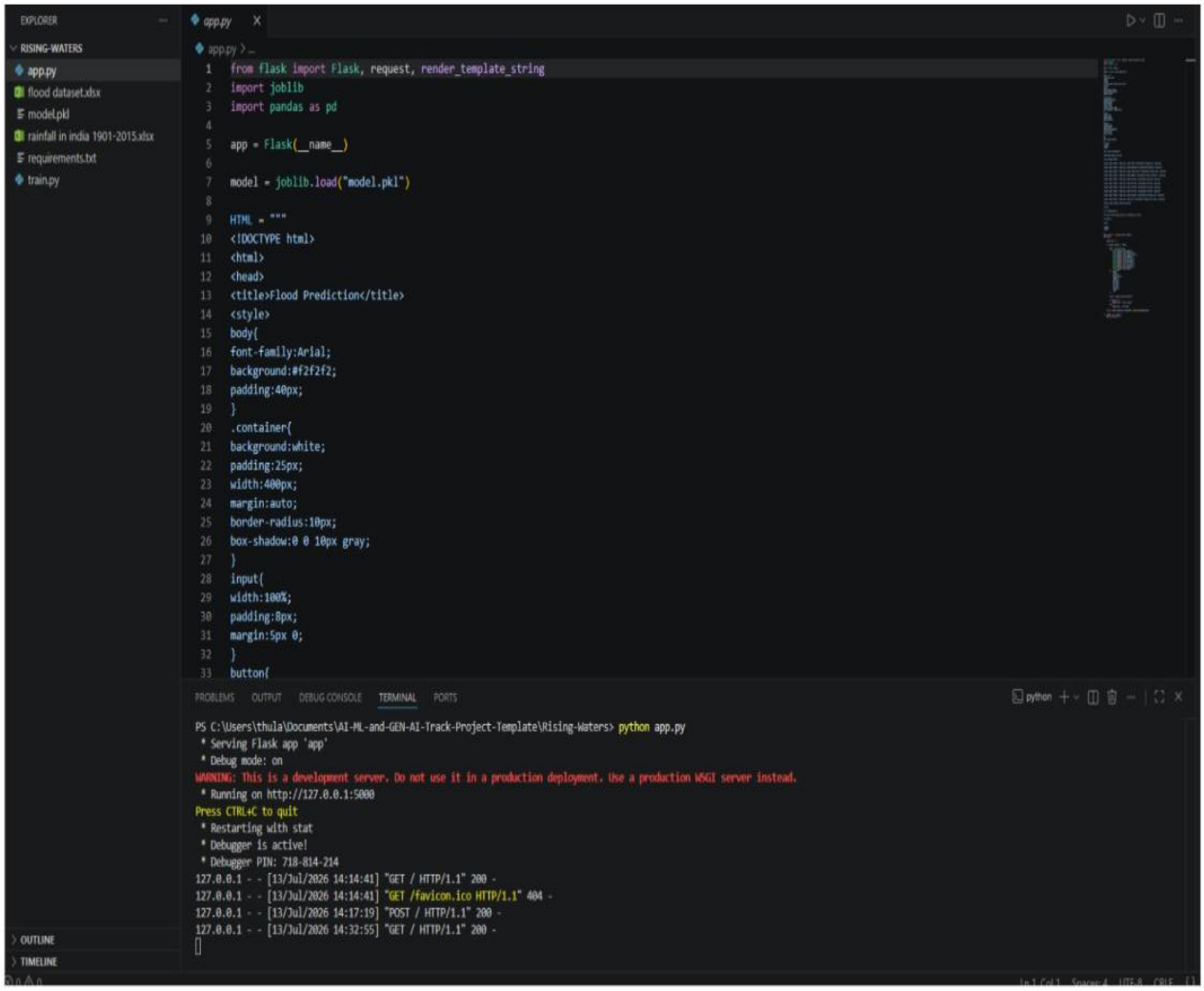
- Define Routes in Flask
- Create the main route (/) in app.py to display the Flood Prediction web page.
- Handle POST requests when the user submits the prediction form.
- Process User Input
- Create an HTML form for entering weather and rainfall parameters.
- Retrieve user input using Flask's request.form.
- Validate and preprocess the input before passing it to the prediction model.
- Integrate Machine Learning Model
- Load the trained model.pkl using the Joblib library.
- Convert user input into the required feature format.
- Pass the processed data to the Random Forest model.
- Display the prediction result ("Flood Likely" or "No Flood") on the web page.Milestone 3: App.py Development

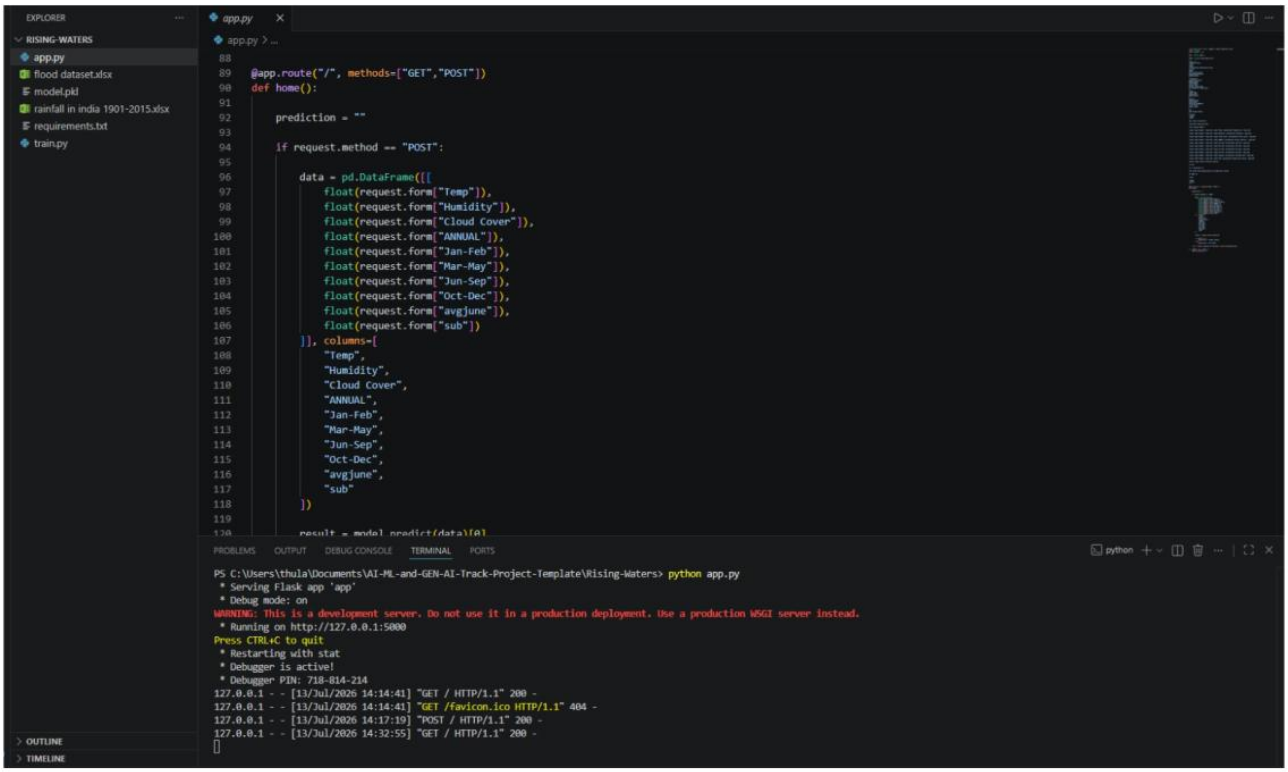
Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the app.py file, which serves as the backend of the Flood Prediction application. In this milestone, the Flask application receives user input, processes the data, loads the trained machine learning model, predicts flood occurrence, and displays the prediction result.

Activity 3.1: Writing the Main Application Logic in app.py

- Define the Core Routes in app.py
-
- Establish the main Flask route for the Flood Prediction application.
-
- / — Home page route that renders the Flood Prediction interface (index.html)./**generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content





Set Up Route Handlers for Flood Prediction :

- Create the main Flask route (/) to handle both GET and POST requests.
- Read user inputs including Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, Average June, and Subdivision using request.form.
- Convert the received values into a Pandas DataFrame with the required feature columns.
- Load the trained Random Forest model (model.pkl) and generate the flood prediction.
- Display the prediction result as "Flood Likely" or "No Flood" on the web page.
- Return the updated HTML page with the prediction result using render_template_string().

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import joblib

# Load dataset
df = pd.read_excel("flood dataset.xlsx")

# Features
X = df.drop("flood", axis=1)

# Target
y = df["flood"]

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train Model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, pred)

print("Model Accuracy:", accuracy)

# Save Model
joblib.dump(model, "model.pkl")

print("Model saved successfully as model.pkl")
```

- **Select the weather and rainfall attributes as input features.**
- **Use Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, Average June, and Subdivision as the model input.**
- **Use the Flood column as the target output for classification.**

```python
# Features
X = df.drop("flood", axis=1)

# Target
y = df["flood"]

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Model Training & Saving: Trains the Random Forest Classifier using the prepared dataset, evaluates the model using prediction accuracy, and saves the trained model as **model.pkl** using Joblib for use in the Flask application.

```python
# Train Model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, pred)

print("Model Accuracy:", accuracy)

# Save Model
joblib.dump(model, "model.pkl")

print("Model saved successfully as model.pkl")
```

Model Prediction and Result Generation

- Accept user input through the Flask web form using **request.form**.
- Convert the entered values into a **Pandas DataFrame** matching the model's feature columns.
- Load the trained **Random Forest** model (model.pkl) and generate the prediction using model.predict().

```
119
120         result = model.predict(data)[0]
121
122         if result == 1:
123             prediction = "Flood Likely"
124         else:
125             prediction = "No Flood"
126
127     return render_template_string(HTML, prediction=prediction)
128
```

- Validate that all user inputs are received correctly and converted into the required feature format before passing them to the trained Random Forest model for prediction**.**

- Home Route (/) → Renders the Flood Prediction webpage, accepts user inputs through the form (POST), predicts flood occurrence using the trained model, and displays the result as "Flood Likely" or "No Flood".

Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, user-friendly web interface for the Flood Prediction System. The interface allows users to enter weather and rainfall parameters, submit the data, and instantly view whether a flood is likely to occur using the trained machine learning model.**Activity 4.1: Designing and Developing the User Interface**

Set Up the Base HTML Structure:

- Develop the main HTML page for the Flood Prediction application using Flask.
- Create input fields for Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, Average June and Subdivision.
- Add a Predict button to submit user inputs.
- Display the prediction result (Flood Likely / No Flood) below the form.

Design a Responsive Layout Using CSS:

- Design a clean and responsive interface using HTML and CSS.
- Arrange input fields vertically for easy data entry.
- Style the Predict button and prediction result section.
- Ensure compatibility across desktop and laptop screens.

Create Separate UI Components for Each Output Type:

- Input Form: Collect weather and rainfall parameters.
- Prediction Result: Display Flood Likely or No Flood.
- Responsive Layout: Simple, readable interface with proper spacing.

Activity 4.2: Processing User Input and Displaying Prediction]

Handle Form Submission :

- Accept user input through the HTML form and submit the data using the **POST** method.
- Validate the entered weather and rainfall parameters before processing.
- Pass the input values to the Flask backend for flood prediction.
- Load the trained **Random Forest** model and generate the prediction.
- Display the prediction result (**Flood Likely** or **No Flood**) on the same webpage without navigating to another page.

Render Prediction Result:

- Display the prediction result (**Flood Likely** or **No Flood**) immediately after the user submits the form.
- Show the prediction below the input form in a clear and highlighted format.
- Ensure the result updates correctly for every new prediction request.

Restore Application State :

- Clear previous prediction data before processing a new request.
- Keep all input fields available for additional predictions without restarting the application.
- Allow users to modify input values and generate new predictions multiple times.
- Ensure the application continues running without errors after each prediction.

Milestone 5: Deployment

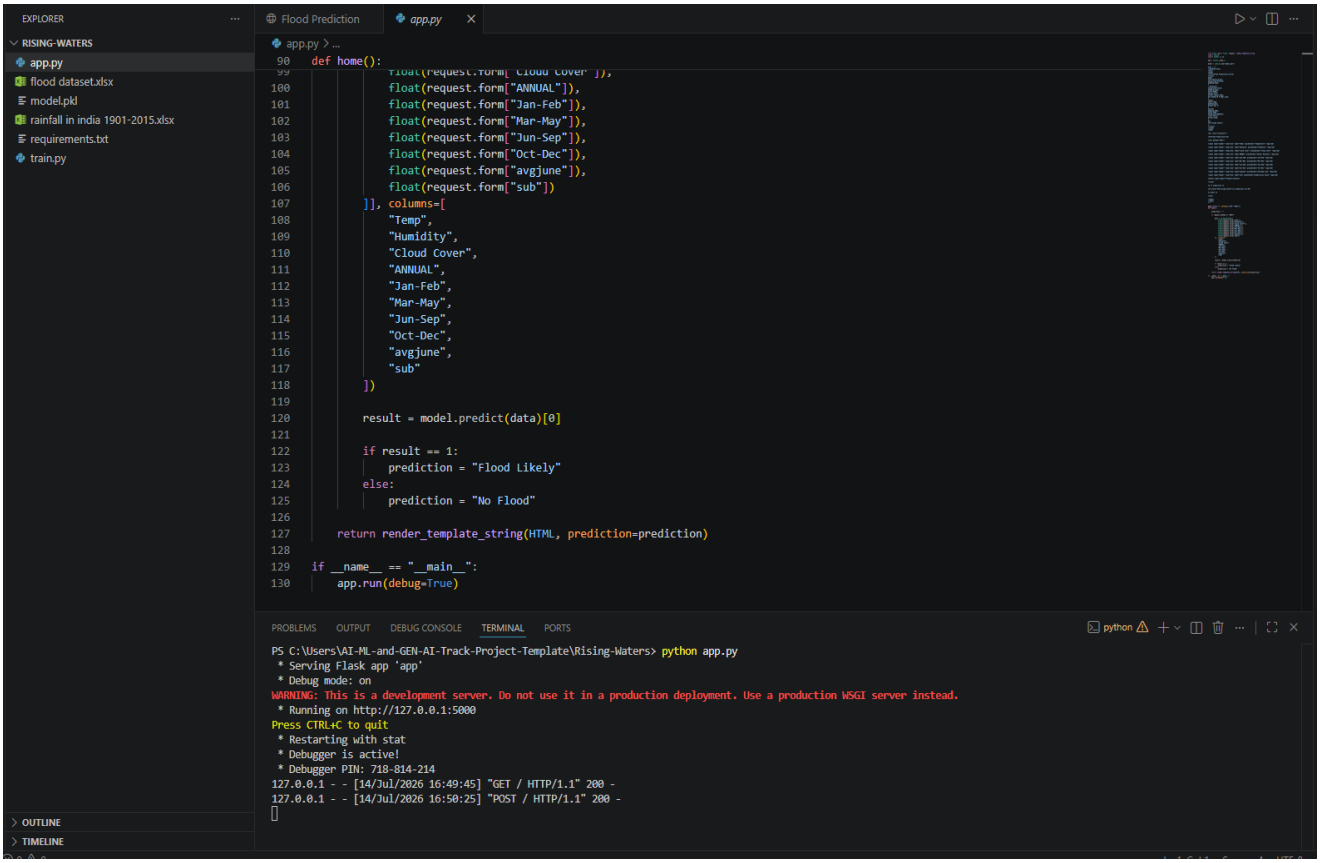
In Milestone 5, the Flood Prediction application is prepared for local deployment using Flask. The trained Random Forest model, project files, and required Python dependencies are configured so that the application can run successfully on a local machine and be accessed through a web browser.

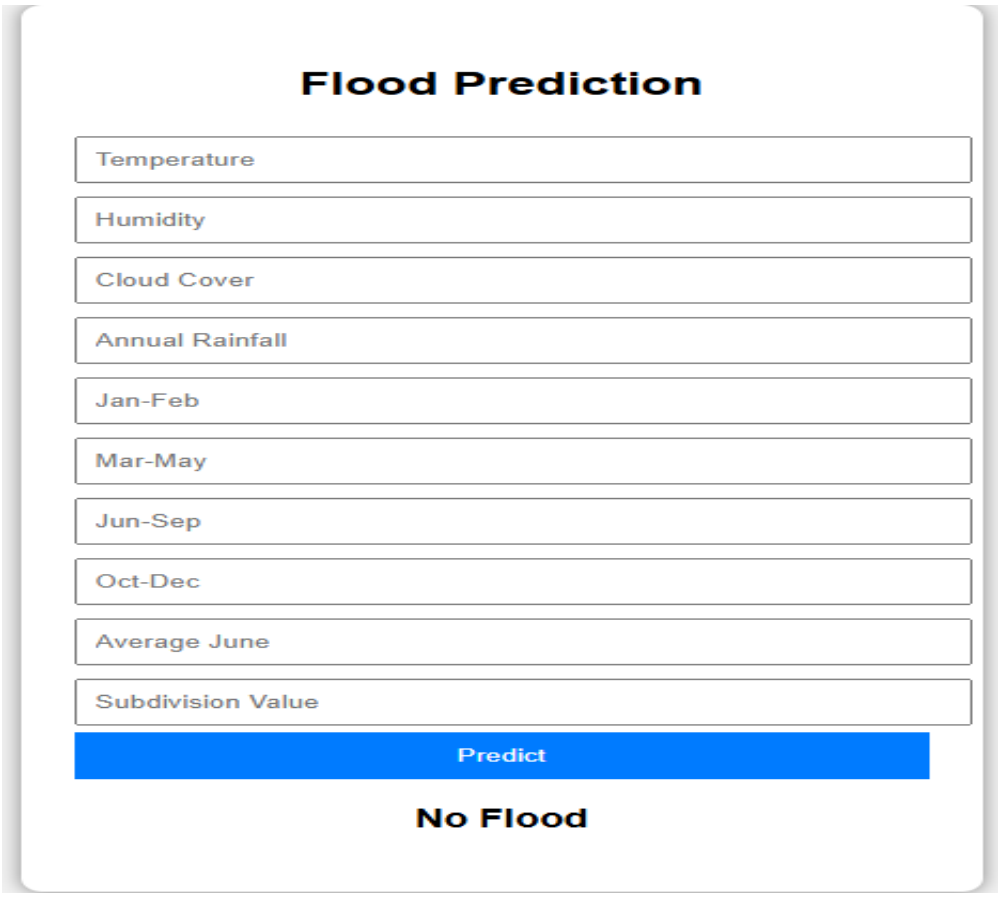
Activity 5.1: Preparing the Application for Local Deployment

Set Up the Development Environment :

- Open the project folder in Visual Studio Code.
- Open the integrated Terminal.
- Install the required Python packages using requirements.txt.
- Ensure the trained model (model.pkl) and dataset files are present in the project folder.
- Run the Flask application using:
-
- Open the application in a web browser using:

```
http://127.0.0.1:5000
```




Flood Prediction
Temperature
Humidity
Cloud Cover
Annual Rainfall
Jan-Feb
Mar-May
Jun-Sep
Oct-Dec
Average June
Subdivision Value
Predict
No Flood

Activity 5.3: Running the Application Locally

The Flood Prediction application is executed locally using the Flask development server. After starting the server, the application becomes accessible through a web browser at http://127.0.0.1:5000, where users can enter rainfall and weather parameters to predict flood occurrence.

- Open the project folder in Visual Studio Code.
- Run the Flask application using:

```
python app.py
```

Wait until the terminal displays:

- Running on http://127.0.0.1:5000

```
if request.method == "POST":

    data = pd.DataFrame([[
        float(request.form["Temp"]),
        float(request.form["Humidity"]),
        float(request.form["Cloud Cover"]),
        float(request.form["ANNUAL"]),
        float(request.form["Jan-Feb"]),
        float(request.form["Mar-May"]),
        float(request.form["Jun-Sep"]),
        float(request.form["Oct-Dec"]),
        float(request.form["avgjune"]),
        float(request.form["sub"])
    ]], columns=[
        "Temp",
        "Humidity",
        "Cloud Cover",
        "ANNUAL",
        "Jan-Feb",
        "Mar-May",
        "Jun-Sep",
        "Oct-Dec",
        "avgjune",
        "sub"
    ])

    result = model.predict(data)[0]
```

Run the Flask Application

- Open the project folder in Visual Studio Code.
- Open the integrated terminal.
- Run the application using:
  -----→python app.py

Wait until the Flask server starts successfully.

Verify the terminal displays:

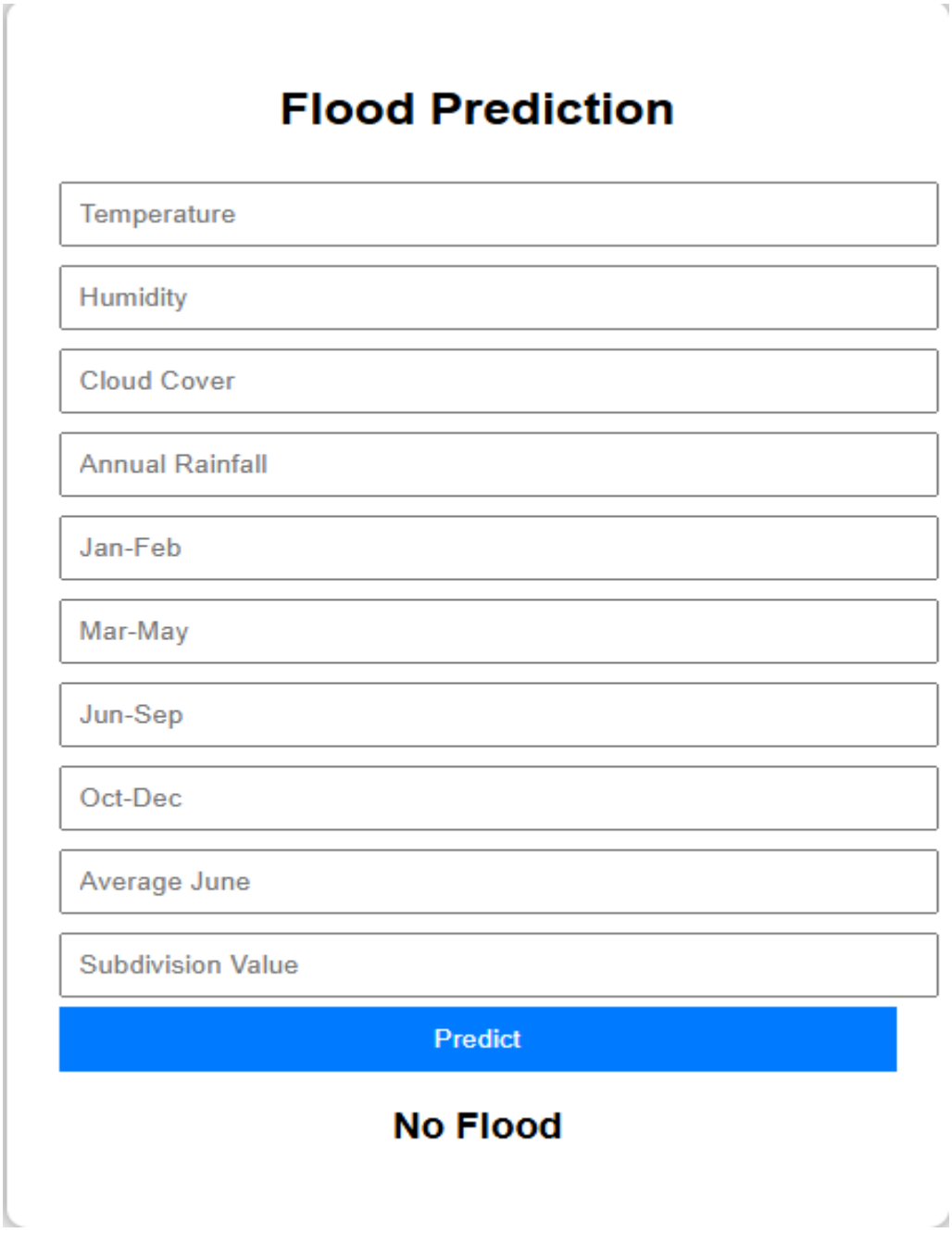
* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5000

Access the Application

- Open any web browser.
- Navigate to:
  ----→http://127.0.0.1:5000
- Enter the required weather and rainfall values.
- Click the **Predict** button.
- View the flood prediction result on the webpage.

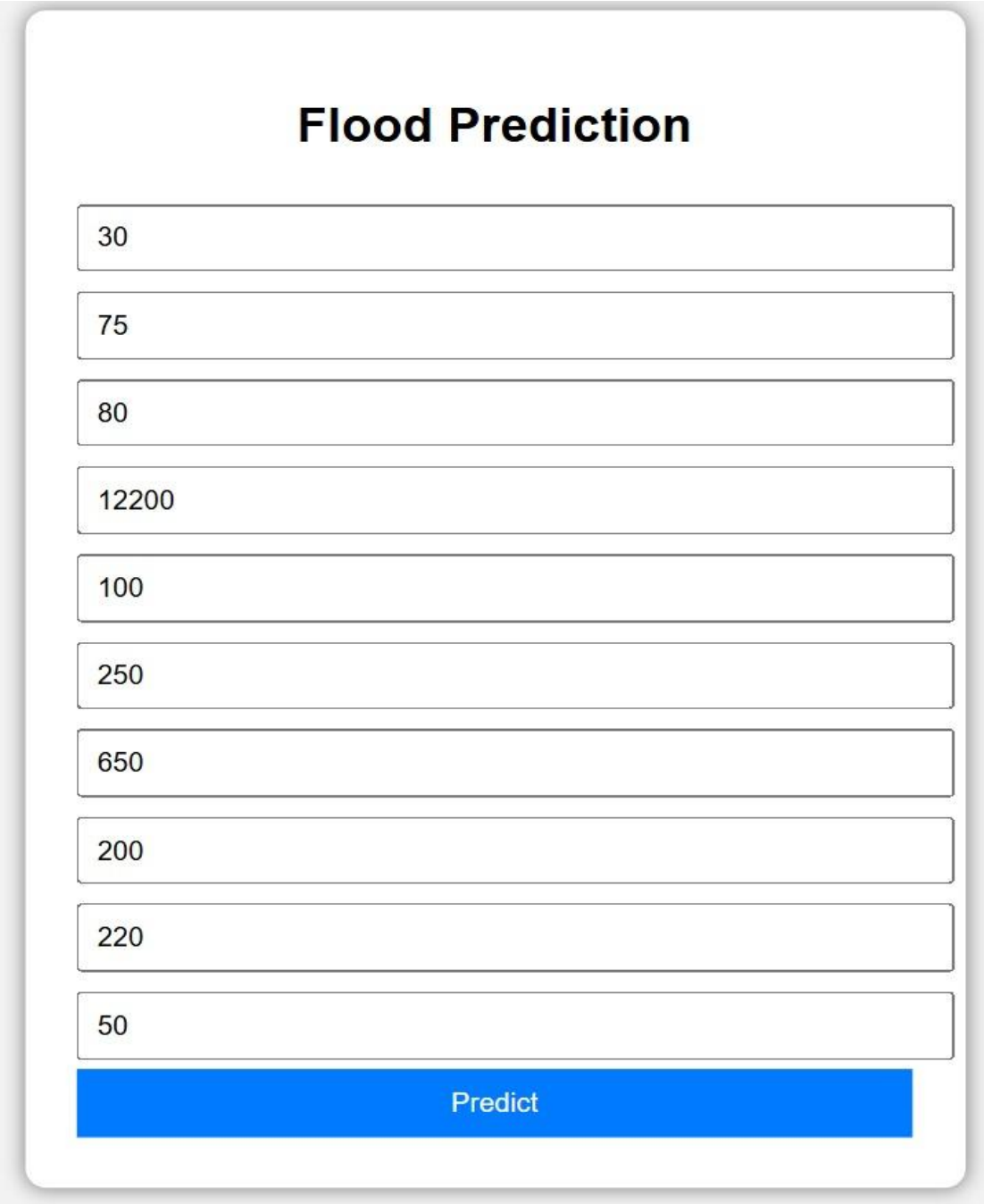
Exploring the Website's Web Pages:

Home / Generator Page:



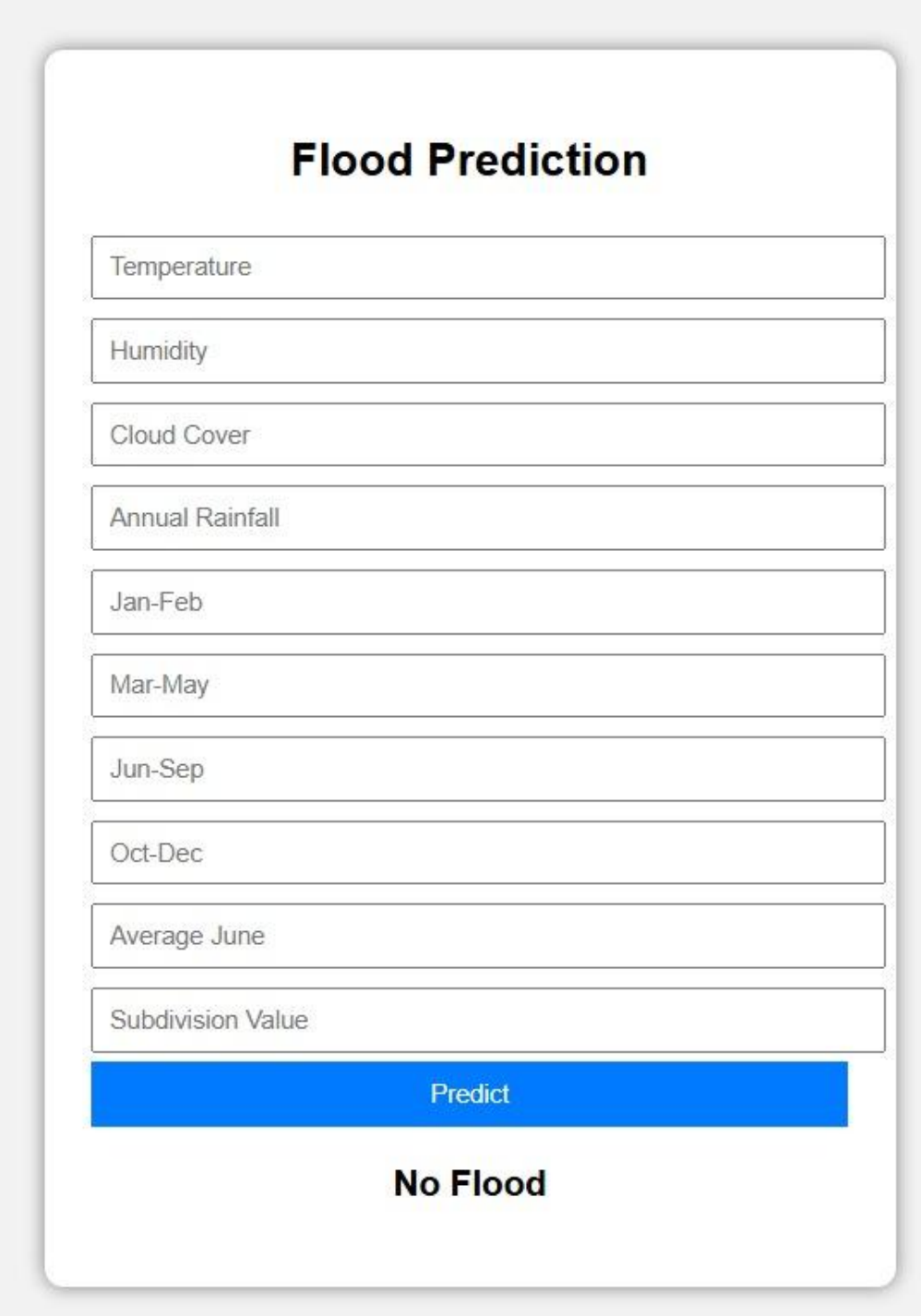
Description: The Flood Prediction application provides a simple web interface for estimating the likelihood of flooding using weather and rainfall-related parameters. Users enter values such as Temperature, Humidity, Cloud Cover, Annual Rainfall, seasonal rainfall (Jan-Feb, Mar-May, Jun-Sep, Oct-Dec), Average June Rainfall, and Subdivision Value. After clicking the **Predict** button, the trained Machine Learning model analyzes the input and displays whether flood conditions are expected.

Input Section:



Description : The input section allows users to enter the weather and rainfall parameters required for flood prediction. The form includes fields for Temperature, Humidity, Cloud Cover, Annual Rainfall, Jan-Feb Rainfall, Mar-May Rainfall, Jun-Sep Rainfall, Oct-Dec Rainfall, Average June Rainfall, and Subdivision Value. After entering all the required values, users click the Predict button to send the data to the trained Random Forest Machine Learning model for analysis.

Results Section:



Description: After the user submits all required weather and rainfall parameters, the application processes the data using the trained **Random Forest Machine Learning model**. The prediction result is displayed instantly below the input form, indicating either **Flood** or **No Flood** based on the provided conditions. This allows users to quickly understand the flood risk associated with the entered environmental data.

Conclusion:

The **Flood Prediction System** is a Machine Learning-based web application developed using **Python, Flask, Pandas, Scikit-learn, HTML, and CSS** to predict the likelihood of floods based on weather and rainfall parameters. The system uses a trained **Random Forest Classifier** to analyze inputs such as Temperature, Humidity, Cloud Cover, Annual Rainfall, seasonal rainfall data, Average June Rainfall, and Subdivision Value, providing users with an instant prediction of **Flood** or **No Flood**.

The project demonstrates how Machine Learning can support early flood risk assessment through a simple and user-friendly interface. Future enhancements may include integrating real-time weather data, GIS-based visualization, improved prediction accuracy using larger datasets, mobile application support, and cloud deployment for wider accessibility. These improvements will make the system more reliable and useful for disaster preparedness and decision-making.